

GROWDEV

Formações Técnicas

Desafio Final Integrador

GrowStore

Loja Oficial Growdev & Growlabs

Trilhas contempladas:

Desenvolvimento Front-End com ReactJS e VTEX IO

Desenvolvimento Back-End com NodeJS e C#

Desenvolvimento Mobile com Flutter

2025

1. Apresentação do Desafio

O Desafio Final Integrador GrowStore é o projeto de conclusão das formações técnicas da Growdev. Inspirado no protótipo de e-commerce disponível em

<https://growstore-hmqdvdyd3.manus.space/>

a proposta une três trilhas de formação em torno de uma mesma temática: a criação de um ecossistema digital completo para a GrowStore, loja oficial de produtos exclusivos das marcas Growdev e Growlabs.

Cada trilha desenvolve uma camada diferente desse ecossistema:

Trilha	Produto a Desenvolver	Tecnologias
Front-End VTEX IO	Loja GrowStore na VTEX IO	ReactJS, Store Framework, Node.js VTEX
Back-End Node.js e C#	API GrowStore	Node.js / C# (.NET 8), SQL, Docker
Mobile Flutter	App Mobile GrowStore	Flutter, Dart, MobX, Firebase

Embora cada turma trabalhe de forma independente, os produtos devem ser concebidos como partes complementares de um mesmo ecossistema. O aluno deve compreender seu papel no contexto maior, demonstrando visão de produto e capacidade de colaboração interdisciplinar.

2. Contexto da GrowStore

A GrowStore é a loja oficial de produtos customizados das marcas Growdev e Growlabs, voltada para quem vive e respira tecnologia. O catálogo inclui:

- Canecas e copos térmicos

- Camisetas e moletons
- Mochilas e bolsas
- Garrafas térmicas
- Blocos de nota e canetas
- Adesivos e mousepads

O protótipo de referência (disponível na URL acima) serve como guia visual e funcional para todos os três produtos. Cada equipe deve utilizá-lo para entender a proposta de valor, identidade visual e fluxos esperados pelo usuário final.

3. Regras Gerais — Aplicáveis a Todas as Trilhas

3.1 Entrega e Repositório

- O projeto deve ser entregue via repositório público no GitHub.
- O repositório deve conter um arquivo README.md completo com: descrição do projeto, tecnologias utilizadas, instruções de instalação e execução, e screenshots ou vídeo demonstrativo.
- O branch principal (main ou master) deve sempre conter a versão estável e entregável do projeto.
- O histórico de commits deve refletir o progresso real do desenvolvimento (mínimo de 10 commits com mensagens semânticas e descritivas).
- Commits semânticos são obrigatórios: feat, fix, chore, docs, refactor, test, style.

3.2 Qualidade de Código

- O código deve ser organizado, legível e seguir as convenções da linguagem/framework utilizado.
- Não são permitidos blocos de código comentado sem justificativa ou arquivos de configuração com credenciais expostas.
- Variáveis de ambiente sensíveis devem ser gerenciadas com arquivos .env (não versionados) e documentadas no README.
- O projeto deve funcionar corretamente seguindo as instruções do README. Projetos que não executam na avaliação serão desconsiderados.

3.3 Apresentação Final

- O aluno deve realizar uma apresentação ao vivo (ou gravada) do projeto, demonstrando as funcionalidades implementadas.
- A apresentação deve cobrir: visão geral do produto, decisões técnicas relevantes, dificuldades encontradas e como foram superadas.
- O tempo de apresentação é de 10 a 15 minutos.

3.4 Prazo e Entrega

Prazo de entrega	A ser definido pelo mentor/coordenação de cada turma
Forma de entrega	Link do repositório GitHub enviado pela plataforma da Growdev
Apresentação	Data e formato definidos pela coordenação de cada trilha
Trabalho individual ou em grupo	A critério do mentor responsável por cada turma

TRILHA 01

4. Desafio Front-End — Loja GrowStore na VTEX IO

Trilha	Desenvolvimento Front-End com ReactJS e VTEX IO
Produto	Loja GrowStore completa na plataforma VTEX IO
Tecnologias obrigatórias	VTEX IO, Store Framework, ReactJS, TypeScript, Node.js (APIs VTEX)
Referência visual	https://growstore-hmqdvdyd3.manus.space/

4.1 Objetivo

Desenvolver uma loja virtual completa para a GrowStore na plataforma VTEX IO, utilizando o Store Framework para montagem de páginas, componentes React customizados e integração com a camada de back-end serverless da VTEX. O produto final deve ser funcional, responsivo e fiel à identidade visual da GrowStore.

4.2 Requisitos Funcionais

4.2.1 Estrutura e Navegação

- Header personalizado com logo GrowStore, barra de busca funcional e minicart.
- Footer com links institucionais, redes sociais e informações de contato.
- Menu de navegação com categorias do catálogo (Vestuário, Acessórios, Papelaria etc.).
- Responsividade garantida para desktop, tablet e mobile.

4.2.2 Home Page

- Banner principal rotativo com pelo menos 2 banners configuráveis via Site Editor.
- Vitrine de produtos em destaque (Shelf) com no mínimo 4 produtos.
- Seção de categorias com links visuais para as principais categorias.
- Banners secundários configuráveis para campanhas promocionais.

4.2.3 Página de Lista de Produtos (PLP)

- Listagem de produtos por categoria com filtros e ordenação funcionais.
- Exibição de produto com imagem, nome, preço e botão de adicionar ao carrinho.
- Paginação ou scroll infinito.

4.2.4 Página de Detalhe de Produto (PDP)

- Galeria de imagens do produto.
- Seletor de variações (SKU) como cor e tamanho.
- Exibição de preço, descrição e disponibilidade em estoque.
- Botão de adicionar ao carrinho com feedback visual.
- Seção de produtos relacionados.

4.2.5 Carrinho e Checkout

- Minicart funcional com listagem de itens, quantidade e subtotal.
- Fluxo de checkout configurado e operacional em ambiente de teste.
- Configuração de ao menos um método de pagamento simulado.

4.2.6 Catálogo e CMS

- Cadastro de no mínimo 8 produtos reais com imagens, descrições, preços e SKUs no painel VTEX.
- Organização em pelo menos 3 categorias distintas.
- Configuração de banners e conteúdos via Site Editor (sem necessidade de publicação de código).

4.2.7 Componente React Customizado

- Desenvolvimento de ao menos 1 componente React customizado integrado ao Store Framework.
- O componente deve exibir dados dinâmicos (ex.: banner de contagem regressiva, lista de benefícios, seção de depoimentos ou similar).
- O componente deve ter Schema para configuração via Site Editor.

4.3 Requisitos Técnicos

Obrigatórios
Projeto configurado como Store Theme no VTEX IO com workspace dedicado
Uso de CSS Handles para toda customização visual (sem sobrescrever CSS global)
Componentes React em TypeScript
Versionamento semântico e uso do VTEX CLI para deploy

README com instruções de configuração do workspace e instalação do tema

Commits semânticos no repositório Git

Diferenciais (pontuação extra)

Criação de API customizada (Node.js VTEX IO) integrada ao front-end

Internacionalização de componentes customizados (pt-BR / en-US)

Implementação de App Admin com settings configuráveis

Configuração de e-mail transacional customizado

Relatório de performance Lighthouse com pontuação acima de 80

4.4 Critérios de Avaliação

Critério	Peso	Descrição
Funcionalidade	30%	Todas as páginas e fluxos funcionam conforme os requisitos
Qualidade Visual	20%	Fidelidade à identidade GrowStore, responsividade e UX
Código e Boas Práticas	20%	Organização, CSS Handles, TypeScript, commits semânticos
Store Framework	15%	Uso correto de blocos, props, JSON Schema e Site Editor
Catálogo e CMS	10%	Produtos, categorias e banners configurados corretamente
Apresentação	5%	Clareza na demonstração e justificativa das decisões técnicas

4.5 Restrições

- Não é permitido usar temas prontos de terceiros como base — o Store Theme deve ser desenvolvido do zero ou a partir do boilerplate oficial da VTEX.
- Não é permitido sobrescrever estilos via CSS global sem uso de CSS Handles.
- O workspace deve estar acessível para avaliação durante o período de correção.

TRILHA 02

5. Desafio Back-End — API GrowStore

Trilha	Desenvolvimento Web Back-End com Node.js e C#
Produto	API RESTful completa para o ecossistema GrowStore
Tecnologias obrigatórias	Node.js com TypeScript (opção A) ou C# com .NET 8 (opção B)
Banco de dados	Relacional (PostgreSQL ou SQL Server)
Referência de domínio	https://growstore-hmqdvdyd3.manus.space/

5.1 Objetivo

Desenvolver uma API RESTful robusta que sirva como back-end do ecossistema GrowStore. A API deve gerenciar o catálogo de produtos, autenticação de usuários, carrinho de compras e pedidos, seguindo boas práticas de arquitetura, segurança e qualidade de software. O aluno pode escolher entre Node.js com TypeScript ou C# com .NET 8 como stack principal.

5.2 Domínio da Aplicação

A API deve modelar e expor os seguintes recursos do domínio GrowStore:

Recurso	Descrição
Usuários (Users)	Cadastro, autenticação e perfil de clientes
Produtos (Products)	Catálogo com SKUs, variações, preços e estoque
Categorias (Categories)	Organização hierárquica do catálogo
Carrinho (Cart)	Gerenciamento de itens antes do checkout
Pedidos (Orders)	Histórico e status de compras realizadas
Endereços (Addresses)	Endereços de entrega vinculados ao usuário

5.3 Requisitos Funcionais

5.3.1 Autenticação e Usuários

- Cadastro de usuário com nome, e-mail e senha (senha criptografada com bcrypt).
- Login com retorno de JWT (access token).
- Refresh token para renovação de sessão.
- Rota de perfil autenticada retornando dados do usuário logado.
- Middleware de autenticação protegendo rotas privadas.

5.3.2 Catálogo — Produtos e Categorias

- CRUD completo de categorias (criação, listagem, atualização, exclusão).
- CRUD completo de produtos com: nome, descrição, preço, imagens (URL), estoque e categoria.
- Suporte a variações de produto (ex.: tamanho P/M/G/GG ou cor).
- Listagem de produtos com filtro por categoria, faixa de preço e busca por nome.
- Paginação obrigatória em listagens (page e limit como query params).

5.3.3 Carrinho de Compras

- Adicionar item ao carrinho (produto + variação + quantidade).
- Atualizar quantidade de item no carrinho.
- Remover item do carrinho.
- Visualizar carrinho do usuário autenticado com subtotal calculado.

5.3.4 Pedidos

- Criação de pedido a partir do carrinho (checkout simulado).
- Listagem de pedidos do usuário autenticado.
- Detalhe de pedido com itens, valores e status.
- Atualização de status do pedido (PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED) — rota restrita a administradores.

5.3.5 Administração

- Rota de gerenciamento de estoque (atualizar quantidade disponível por produto/SKU).
- Diferenciação de roles: ADMIN e CUSTOMER com controle de acesso por middleware.

5.4 Requisitos Técnicos

5.4.1 Opção A — Node.js com TypeScript

Obrigatórios (Node.js)

TypeScript estrito (strict: true no tsconfig)

Express com arquitetura em camadas (routes, controllers, services, repositories)

PrismaORM para persistência com PostgreSQL

Docker Compose com containers para API e banco de dados

Autenticação JWT com refresh token

Documentação da API com Swagger/OpenAPI

Testes automatizados: mínimo de 10 testes unitários na camada service

Variáveis de ambiente com .env (não versionado) e .env.example documentado

5.4.2 Opção B — C# com .NET 8

Obrigatórios (.NET 8)

Arquitetura em camadas: API, Application, Domain, Infrastructure

Entity Framework Core 8 com SQL Server

Docker Compose com containers para API e banco de dados

Autenticação JWT com roles e claims

Validações com FluentValidation

Documentação com Swagger/Scalar

Testes automatizados: mínimo de 10 testes unitários com xUnit

Variáveis de ambiente via appsettings.json + User Secrets (não versionado)

5.5 Diferenciais

Pontuação Extra

Implementação de testes de integração além dos unitários

Rate limiting e proteção contra ataques de força bruta

Background service para envio de e-mail de confirmação de pedido

Versionamento de API (/v1/...)

Health check endpoint

Cache em memória para listagem de produtos

Geração de relatório de cobertura de testes (acima de 70%)

5.6 Critérios de Avaliação

Critério	Peso	Descrição
Funcionalidade	30%	Todos os endpoints funcionam conforme especificação
Arquitetura e Organização	20%	Separação de responsabilidades, padrões, SOLID/Clean Code
Segurança	15%	Autenticação JWT, proteção de rotas, criptografia de senhas
Banco de Dados	15%	Modelagem, migrations, relacionamentos e queries corretos
Testes Automatizados	10%	Cobertura e qualidade dos testes unitários
Documentação	5%	README, Swagger/OpenAPI e organização do repositório
Apresentação	5%	Demonstração clara das funcionalidades e decisões técnicas

5.7 Restrições

- Não é permitido expor senhas, tokens ou chaves de API no repositório.
- O banco de dados deve ser inicializado automaticamente via migrations — não é permitido entregar dumps SQL como única forma de configuração.
- A API deve estar funcional e executável via docker-compose up para avaliação.

TRILHA 03

6. Desafio Mobile — App GrowStore

Trilha	Desenvolvimento Mobile com Flutter
Produto	Aplicativo mobile da GrowStore para Android e iOS
Tecnologias obrigatórias	Flutter, Dart, MobX (gerenciamento de estado)
Recursos adicionais	Firebase Auth, Google Maps (opcional), armazenamento local
Referência de domínio	https://growstore-hmqdvdyd3.manus.space/

6.1 Objetivo

Desenvolver um aplicativo mobile nativo para Android e iOS utilizando Flutter, que permita aos clientes da GrowStore navegar pelo catálogo de produtos, gerenciar favoritos, realizar o checkout e acompanhar seus pedidos. O app deve ser fluido, responsivo, seguir as diretrizes de UX mobile e utilizar MobX para gerenciamento de estado global.

6.2 Requisitos Funcionais

6.2.1 Autenticação

- Tela de splash screen com logo GrowStore.
- Tela de login com e-mail e senha.
- Tela de cadastro com nome, e-mail, senha e confirmação de senha.
- Login social com Google via Firebase Authentication.
- Logout com limpeza de estado global.
- Persistência da sessão entre aberturas do app (token salvo localmente com Hive).

6.2.2 Catálogo de Produtos

- Tela inicial (Home) com banner rotativo e lista de produtos em destaque.
- Tela de listagem por categoria com filtro e busca por nome.

- Tela de detalhe do produto com galeria de imagens, variações (tamanho/cor), preço e descrição.
- GridView e ListView alternativos para visualização da listagem.
- Scroll infinito ou paginação na listagem.

6.2.3 Carrinho de Compras

- Adicionar produto ao carrinho com seleção de variação e quantidade.
- Tela de carrinho com lista de itens, subtotal e botão de finalizar compra.
- Atualização de quantidade e remoção de itens com gesto Dismissible.
- Animação de feedback ao adicionar item ao carrinho.

6.2.4 Favoritos

- Funcionalidade de adicionar/remover produto dos favoritos.
- Tela de lista de favoritos.
- Persistência dos favoritos entre sessões (armazenamento local com Hive).

6.2.5 Perfil e Pedidos

- Tela de perfil com dados do usuário logado.
- Histórico de pedidos com status (PENDING, PROCESSING, SHIPPED, DELIVERED).
- Tela de detalhe do pedido com itens e valores.
- Gerenciamento de endereços de entrega.

6.2.6 Integração com API

- Todas as telas devem consumir dados reais de uma API REST (pode ser a API desenvolvida pela trilha de Back-End, uma API simulada com MockAPI, ou JSON Server).
- Tratamento de estados de carregamento (loading), erro e lista vazia em todas as telas de dados.
- Autenticação via JWT integrada ao app.

6.3 Requisitos Técnicos

Obrigatórios

Gerenciamento de estado global com MobX (stores separadas por feature)

Injeção de dependência com GetIt

Armazenamento offline com Hive (favoritos e token de sessão)

Arquitetura por feature (lib/features/auth, lib/features/catalog etc.)

Firebase Authentication configurado para login com Google
 Navegação com rotas nomeadas
 Tema claro (light) com identidade visual GrowStore — paleta verde e branco
 Tratamento global de erros e estados de carregamento
 README com instruções de configuração (Firebase, variáveis de ambiente, execução)

6.4 Diferenciais

Pontuação Extra

Tema escuro (dark mode) com alternância via configuração
 Tela de rastreamento de pedido com Google Maps mostrando localização simulada
 Push notifications via Firebase Cloud Messaging (ex.: atualização de status do pedido)
 Testes unitários com mocks: mínimo de 8 testes cobrindo stores MobX e services
 Animações explícitas além das implícitas exigidas
 Cache de imagens com `optimized_cached_network_image`
 Internacionalização: pt-BR e en-US

6.5 Critérios de Avaliação

Critério	Peso	Descrição
Funcionalidade	30%	Fluxos completos: login, catálogo, carrinho, favoritos, perfil
UX e Design Visual	20%	Identidade GrowStore, responsividade, animações e fluidez
Gerenciamento de Estado	15%	Uso correto do MobX com stores bem definidas
Arquitetura e Organização	15%	Estrutura por feature, separação de responsabilidades
Integração com API	10%	Consumo real de API, tratamento de loading/erro
Código e Boas Práticas	5%	Widgets componentizados, Dart idiomático, linting
Apresentação	5%	Demonstração no emulador/dispositivo e decisões técnicas

6.6 Restrições

- O app deve ser executável em emulador Android ou iOS durante a apresentação.
- Não é permitido usar pacotes que abstraíam completamente o Flutter (ex.: FlutterFlow gerado automaticamente).
- Credenciais do Firebase não devem ser versionadas — usar variáveis de ambiente ou arquivos de configuração ignorados pelo Git.

7. Integração Entre Trilhas

Embora cada trilha seja avaliada de forma independente, o desafio foi concebido para que os produtos se integrem. As equipes são encorajadas a colaborar da seguinte forma:

Trilha Produtora	Trilha Consumidora	Ponto de Integração
Back-End (API GrowStore)	Front-End VTEX IO	API customizada no Node.js VTEX IO pode consumir a API GrowStore externa
Back-End (API GrowStore)	Mobile Flutter	O app Flutter consome diretamente os endpoints da API GrowStore
Front-End VTEX IO	Mobile Flutter	Identidade visual, catálogo e fluxos de UX compartilhados como referência

A integração real entre trilhas não é obrigatória, mas é valorizada como diferencial. Em caso de integração, o aluno deve documentar o ponto de integração no README e demonstrá-la na apresentação.

8. Checklist de Entrega

8.1 Checklist Geral (Todas as Trilhas)

- ☐ Repositório público no GitHub com nome descritivo
- ☐ README.md completo (descrição, tecnologias, instalação, execução, screenshots)
- ☐ Mínimo de 10 commits semânticos e descritivos
- ☐ Nenhuma credencial ou token exposto no repositório
- ☐ Projeto executável seguindo as instruções do README

8.2 Checklist Front-End VTEX IO

- ☐ Store Theme configurado com workspace dedicado
- ☐ Home, PLP e PDP implementadas
- ☐ Header e footer customizados

- ☐ Mínimo de 8 produtos no catálogo com 3 categorias
- ☐ Site Editor com ao menos 1 banner configurável
- ☐ Componente React customizado com Schema
- ☐ CSS via CSS Handles (sem override global)

8.3 Checklist Back-End

- ☐ Endpoints de autenticação (cadastro, login, refresh token)
- ☐ CRUD de produtos e categorias
- ☐ Carrinho de compras
- ☐ Pedidos com atualização de status
- ☐ Autorização por roles (ADMIN / CUSTOMER)
- ☐ docker-compose funcional
- ☐ Swagger/OpenAPI disponível
- ☐ Mínimo de 10 testes unitários

8.4 Checklist Mobile Flutter

- ☐ Splash, login (e-mail/senha + Google) e cadastro
- ☐ Home com banner e produtos em destaque
- ☐ Listagem por categoria com busca e filtro
- ☐ Tela de detalhe do produto
- ☐ Carrinho com Dismissible e subtotal
- ☐ Favoritos persistidos com Hive
- ☐ Histórico de pedidos
- ☐ MobX com stores por feature
- ☐ Tema claro com identidade GrowStore

9. Recursos e Referências

9.1 Links Úteis

Recurso	URL
Protótipo GrowStore	https://growstore-hmqdvdyd3.manus.space/
Documentação VTEX IO	https://developers.vtex.com/docs/guides/vtex-io-documentation
Store Framework	https://developers.vtex.com/docs/guides/store-framework
VTEX CLI	https://developers.vtex.com/docs/guides/vtex-io-documentation-vtex-io-cli-installation-and-command-reference

PrismaORM	https://www.prisma.io/docs
.NET 8 Docs	https://learn.microsoft.com/pt-br/aspnet/core
Flutter Docs	https://docs.flutter.dev
MobX Flutter	https://mobx.netlify.app/getting-started
Firebase Flutter	https://firebase.google.com/docs/flutter/setup
Hive (armazenamento local)	https://docs.hivedb.dev

9.2 Ferramentas Recomendadas

- Postman ou Insomnia — testes de API
- DBeaver — gerenciamento de banco de dados
- Docker Desktop — execução de containers
- VS Code — editor universal
- Android Studio / Xcode — emuladores mobile
- VTEX CLI — gerenciamento de workspace e deploy
- Figma — referência visual e prototipação

10. Observações Finais

O Desafio GrowStore representa a consolidação de tudo que foi aprendido ao longo da formação. Ele é propositalmente amplo para que cada aluno possa expressar seu nível de maturidade técnica — o objetivo não é chegar ao produto perfeito, mas demonstrar raciocínio estruturado, domínio das tecnologias estudadas e capacidade de entregar valor real.

Dúvidas sobre os requisitos devem ser levadas ao mentor responsável pela trilha. Adaptações nos requisitos podem ser feitas pelo mentor conforme o contexto e andamento de cada turma.

Bom desafio a todas as turmas!

Growdev — Formando Pessoas para o Mercado de Tecnologia